Kinetic and Thermodynamic Parameters for the Flux Module

Five predictors run, and formation energies recomputed with uncertainty

Michael Volk

September 4, 2026

Abstract

The enzyme-constrained flux layer needs three parameter tables and had almost none of them. Measured k_{cat} covered 148 of 3,728 catalytic units, 4.0 percent, with the other 96 percent running on a single flat default of 10.3 s^{-1}; K_M had no table at all; and the thermodynamics came from a shipped file of point values with no covariance, usable only for this organism. All three are now built, and every value carries the `sha256` of the weights or the model version that produced it.

Six kinetic predictors were mirrored into a standard location with per-file hashes, retrieval commands and git commits, and five were run over the enzyme-substrate pairs of yeast-GEM 9.0.2, with protein sequences taken from the genome object rather than from the table the model bundles. Validating each against its own authors' published examples paid for itself: **two of these models exponentiate base 2 and two base 10**, and reading either wrong yields turnover numbers that still look credible. Two entries in an earlier readiness audit were reversed by reading files rather than filenames, in both directions, and a third blocker dissolved entirely once its dependency was checked.

The five predictors agree on the median k_{cat} and disagree per pair. On identical enzyme-substrate pairs their rank correlation ranges from **0.015 to 0.509**, with a typical disagreement of 0.4 to 0.7 decades. Since almost none of these pairs has a measured value, that spread, and not any published accuracy figure, is the honest uncertainty on a single predicted turnover number.

Formation energies were recomputed from eQuilibrator's component contribution, which returns a covariance and applies to any organism because it works from structure. Summing the resulting table reproduces eQuilibrator's own reaction call to 9.1×10^{-13} kJ mol^{-1}, so the table is correct; on the chemical reactions both tables cover it agrees with the shipped file to a median of 10.11 kJ mol^{-1}, which is about as well as the shipped file agrees with itself. The gain is 785 reactions with a real standard error, median 2.30 kJ mol^{-1}, and the rank-590 covariance factor the flux layer had recorded as unavailable. The cost is reaction coverage, 42.5 percent against 77.7. Compartment-specific pH is implemented and not trusted: it moves 1,309 reactions by a median 42 kJ mol^{-1}, which is not credible.

The predictors agree with each other far less than either agrees with its own training data, and that gap is the honest uncertainty on any single predicted value.

Section status: ✓ ready ■ author-reviewed, keep stable ✗ not done, agents may edit freely

Contents

1 Where the parameters were, and where they are now ✗ . . . **3**
1.1 The starting state ✗ . . . 3
1.2 A standard place for a model, with provenance ✗ . . . 3
1.3 Two corrections to the readiness audit ✗ . . . 3
1.4 One environment per predictor ✗ . . . 3
2 The predictors, and what running them taught ✗ . . . **4**
2.1 One module per model, on the node-embedding pattern ✗ . . . 4
2.2 Sequences come from the genome, not from the model's bundled table ✗ . . . 4
2.3 Validation caught a log base ✗ . . . 4
2.4 Embedding on four GPUs ✗ . . . 4
2.5 What each predictor cost to run ✗ . . . 5
3 Thermodynamics recomputed from eQuilibrator ✗ . . . **5**
3.1 Why the shipped table is a dead end ✗ . . . 5
3.2 Resolution, and what it costs ✗ . . . 5
3.3 The uncertainty, which is the point ✗ . . . 6
3.4 Agreement with the shipped table ✗ . . . 6
3.5 Compartment pH is implemented and not trusted ✗ . . . 7
3.6 The correctness check ✗ . . . 7

3.7 Where the method stops ✗ . . . 8
4 Reproducing Wu et al.'s Fig. 3 ✗ . . . 8
4.1 What the original figure claims ✗ . . . 8
4.2 The per-pair predictions behind Fig. 3 were not released ✗ . . . 8
4.3 The log base is not a convention ✗ . . . 9
4.4 The comparison between predictors, which the original does not make ✗ . . . 10
4.5 The comparison per gene, which is the unit the model reads ✗ . . . 10
5 Where these tables enter the model ✗ . . . 11
5.1 Why these two tables, in two equations ✗ . . . 12
6 What these tables cannot support ✗ . . . 14
6.1 A predicted turnover number is not a measured one ✗ . . . 14
6.2 Accuracy against the Open Enzyme Database is memorization, not generalization ✗ . . . 14
6.3 K_M is built and deliberately not consumed ✗ . . . 14
6.4 The thermodynamic gaps ✗ . . . 14
6.5 What the mirror records when a predictor cannot run ✗ . . . 14
7 Every source, for citation and for the mirror ✗ . . . 15
7.1 Papers ✗ . . . 15
7.2 Code and weights ✗ . . . 15
7.3 Four provenance defects to repair ✗ . . . 15

1 Where the parameters were, and where they are now ✗

1.1 The starting state ✗

Before this work, yeast-GEM 9.0.2 had 3,728 catalytic units and one turnover number for almost none of them. Measured k_{cat} covered 148 units, 4.0 percent, drawn from the Open Enzyme Database's 1,126 *S. cerevisiae* rows, which between them name only 106 distinct UniProt accessions. The other 3,580 units, 96 percent of the model, ran on a single flat default of 10.3 s^{-1}, the median of the values that did resolve. K_M had no table at all: the parameter existed as an enum member and a protocol flag with nothing behind either.

Thermodynamics was in better shape and worse than it looked, which Section 3 takes up.

1.2 A standard place for a model, with provenance ✗

The predictors arrived as ad-hoc git clones in a scratch directory, which is not something a result can be rebuilt from: nothing recorded which commit was cloned, and nothing would detect an upstream force-push. They now live at `$DATA_ROOT/models/kinetics/<name>/`, beside the existing `mineru` mirror, each with a `manifest.json` pinning every file by `sha256` and recording the source URL, the exact retrieval command, and the git commit.

Two trees are kept deliberately apart. `$DATA_ROOT/models/...` holds *the model*: weights, vocabularies, dictionaries, written once and verified thereafter. `$DATA_ROOT/data/torchcell/...` holds *what the model produced*, regenerable from a mirror plus code, and every row carries the `sha256` of the weight file that produced it. Deleting a build is routine; deleting a mirror invalidates the provenance of every build made from it, so a mirror is only ever added.

The reason to pay this cost is that a predicted k_{cat} is a model output, not a measurement, and the difference has to survive into the table. A value with no provenance becomes indistinguishable from a measured one within two steps, and the flux layer's coverage gate has to tell an experimental value, a predicted value and a filled default apart.

1.3 Two corrections to the readiness audit ✗

An earlier audit of these six repositories recorded which ship weights. Reading the files rather than the filenames reversed two of its entries, and both reversals matter.

DLKcat does ship its trained model. It is `Results/output/all--radius2--...--iteration50`, a PyTorch `state_dict` with no file extension, which is why a scan for `*.pt`/`*.pkl` missed it. On the strength of that miss an 18 GB Zenodo download had been queued that was never needed.

Boost_KM ships two fitted models, and the citation points at the wrong one. Its 18 "weight" files are the UniRep 1900 mLSTM *featurizer*, and reading only those supports the conclusion that no regressor was released. Beside them sit `xgboost_model.dat` and `xgboost_model_full.dat`, which are the fitted K_M regressors, and the repository's own README says the pretrained weights can be loaded rather than retrained. Nothing needed refitting.

What Wu et al. ran is a third model again. Their Methods cite `AlexanderKroll/KM_prediction`, the repository for the published paper, but the notebook in their release vendors `KM_prediction_function`, whose enzyme representation is ESM-1b rather than UniRep and whose regressor, `xgboost_model_new_KM_esm1b.dat`, takes 1,332 features rather than 1,952. Kroll's README states the substitution plainly: "the model provided in the repository 'KM_prediction_function' is slighly different to the one presented in our paper: Instead of the UniRep model, we are using the ESM-1b model." The substrate encoder is common to both, and its three checkpoint files are byte-identical across the two repositories, so only the enzyme half and the regressor differ. Reproducing Fig. 3f means following the code rather than the citation.

1.4 One environment per predictor ✗

A released predictor is a frozen artifact, and several of them pin dependencies that cannot coexist. UniKP's regressors were pickled before scikit-learn 1.3 added `missing_go_to_left` to the decision-tree node dtype, and scikit-learn 1.7 refuses to load them. Refusing is correct: reinterpreting a tree's node array under a changed dtype would yield numbers that are not UniKP's predictions.

So UniKP runs in an environment pinned to scikit-learn 1.2.2, invoked as a subprocess with parquet as the interface, and the torchcell-side dataset class calls it rather than importing it. Treating that as an accident to be patched around would be a mistake; it is the shape this has to take, and DeepEnzyme additionally wants NVIDIA apex and an old numpy for the same reason.

2 The predictors, and what running them taught ✗

2.1 One module per model, on the node-embedding pattern ✗

Which predictor feeds the flux layer is a design choice to experiment over, not a pipeline detail to settle once, so the kinetic predictors copy the shape the sequence-embedding datasets already use: an abstract base, one module per model, and a name-to-class registry that a config string selects.

One structural difference drives everything else. A node embedding is keyed by *gene*; a kinetic parameter is keyed by a *pair*, because a turnover number is a property of one enzyme acting on one substrate. The key is (uniprot, substrate, predictor, parameter). Collapsing to one row per gene would silently pick a substrate, and collapsing to one row per catalytic unit would silently pick a subunit, so the store holds pairs and aggregation lives in the resolver. The dataset records what was predicted; the caller decides what the layer sees.

2.2 Sequences come from the genome, not from the model's bundled table ✗

yeast-GEM ships a `swissprot.tsv` with a sequence column, and it sits conveniently beside the model. It is the wrong source. Protein sequences are read from `SCerevisiaeGenome` through `genome[orf].protein.seq`, the same path the ESM2 embedding dataset uses, so everything stays on the derivation chain from the S288C reference FASTA plus the GAF. The difference is provenance rather than content: a sequence from a third-party table is a claim by that table's compiler, while the same sequence from the genome is the one the perturbation ontology edits, so a strain's genotype and the protein a predictor sees cannot silently disagree.

All 1,161 yeast-GEM genes resolve as `current` through the shared gene-name reconciler, and all 1,161 have a protein sequence, so nothing is lost by the switch. Rebuilding DLKcat on genome sequences moved the median k_{cat} from 4.0125 to 4.0116 s^{-1}, which says the two sources agree on content and differ on provenance.

Two failures are worth recording because both are silent. `orf in genome` looks safer than indexing and is not: the genome defines no `__contains__`, so Python falls back to integer indexing and calls `genome[0]`, raising before the real lookup happens. And `go_root` is a separate constructor argument defaulting to a relative path; left unset it misses and the constructor tries to fetch `go.obo` from `current.geneontology.org`, which answers 403.

2.3 Validation caught a log base ✗

The DLKcat implementation reproduces the authors' three released example values to four decimal places. That check was worth more than its cost, because it caught two defects that would otherwise have been invisible in a plausible-looking table.

The output is $\log_2$, not $\log_{10}$. The authors exponentiate with `math.pow(2, ...)`. Reading it as base 10 inflates every turnover number by a power of roughly 3.3 and still produces numbers that look like credible turnover numbers.

Multi-component SMILES are refused by the model itself. Its fingerprint pass indexes a node list built only from bonded atoms, so a disconnected fragment, such as the lone `[Fe+2]` in the heme SMILES yeast-GEM gives for ferricytochrome *c*, walks off the end of the list. Excluding them is what the model does, so the exclusion is recorded rather than worked around.

2.4 Embedding on four GPUs ✗

UniKP's protein half is ProtT5-XL, a 3-billion-parameter encoder, and its pinned environment holds CPU-only torch. The embedding is therefore produced once on GPU and handed across as a content-addressed cache, keyed by a hash of the sequence so that any predictor wanting a ProtT5 representation can reuse it and two ORFs with one sequence resolve to one vector.

Sequences are split by index across the four GPUs and merged; within a shard they are sorted by length and batched, which is where most of the gain comes from, and a per-sequence masked mean keeps the result identical to unbatched inference. The 1,154 distinct sequences finish in under 40 seconds.

The genome is read exactly once, by a dump step, and the shards read that dump. Four processes opening the same gffutils SQLite database concurrently raced and made it briefly unreadable; since the shards need the genome for nothing but a sequence lookup, the dependency was removed rather than serialized.

2.5 What each predictor cost to run ✗

Table 1: The six predictors named in Wu et al.'s Methods, what each emits, and what it took to run here. "Ships weights" refers to the repository as cloned; UniKP's fitted regressors are released separately on HuggingFace and are mirrored alongside it.

Predictor	Emits	Ships weights	Note
DLKcat	k_{cat}	yes, extensionless	output is $\log_2$
UniKP	k_{cat}, K_M	regressors on HuggingFace	needs scikit-learn 1.2.2
TurNuP	k_{cat}	Zenodo archive	takes full reaction SMILES
EITLEM-Kinetics	k_{cat}, K_M	Zenodo, 12.3 GB	many checkpoints
Boost_KM	K_M	yes, two regressors	ESM-1b variant is the one Wu ran
DeepEnzyme	k_{cat}	figshare	uses AlphaFold structures

3 Thermodynamics recomputed from eQuilibrator ✗

3.1 Why the shipped table is a dead end ✗

yeast-GEM 9.0.2 ships `model_metDeltaG.csv` and `model_rxnDeltaG.csv`, and they are usable: 2,389 of 2,806 metabolites and 3,210 of 4,131 reactions carry a value. They also close two doors. They are point values with no covariance, so the correlated error term $\Delta_r G^{\circ,\text{error}} = Qm$ that a thermodynamic flux formulation calls for cannot be formed from them at all. And they exist only for this model, so a table built on them transfers to no other organism.

Component contribution answers both at once. It decomposes a compound into groups, fits group energies against the measured reaction corpus, and returns a formation energy together with the covariance implied by that fit. Applied to a structure it works for any organism, and for compounds no database has tabulated.

Reading the shipped files correctly is itself a trap worth recording. They use *two* missing-value conventions: most gaps are the sentinel 10^7, but a further 51 metabolites and 120 reactions are the literal string `NaN`. Filtering only the sentinel leaves the NaNs, which then propagate through every sum and produce NaN gradients from what looks like a coverage of 87 percent.

3.2 Resolution, and what it costs ✗

Each metabolite is resolved by accession, MetaNetX first, then ChEBI, then KEGG, then BiGG, with an InChI structure lookup as a fallback. Figure 1 shows where they land: MetaNetX carries almost all of it, and 332 metabolites resolve to nothing.

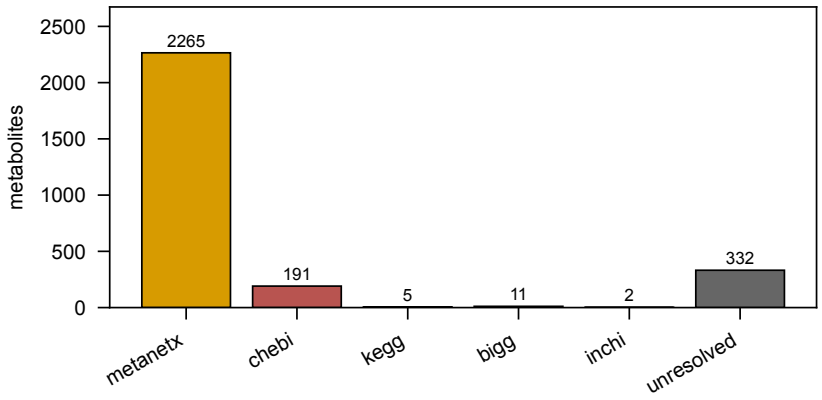


Figure 1: Which identifier namespace resolved each of yeast-GEM's 2,806 metabolites to an eQuilibrator compound, in the order the resolver tries them. A metabolite counted under `chebi` is one whose MetaNetX identifier missed, so the bars are disjoint rather than nested. The 332 unresolved are dominated by acyl-CoA isomers and the `biomass` pseudo-metabolite. Generated by `experiments/026-metabolism-flux/scripts/plot_equilibrator_thermo.py`.

The coverage cost is real and is stated rather than softened. Against the shipped table's 2,389 metabolites and 3,210 reactions, the recomputation covers 2,099 metabolites (74.8 percent) and 1,755 reactions with every participant known (42.5 percent). Buying uncertainty and organism-independence costs reaction coverage, because a reaction needs *all* of its participants resolved before it has an energy at all.

The two tables are not nested, and that changes what to do with them. Treating 42.5 against 77.7 as a straight loss assumes the recomputation covers a subset of what the GEM ships, and it does not:

Table 2: Which of yeast-GEM's 4,131 reactions each table can price, at uniform pH. The recomputation is not a subset of the shipped file.

Priced by	Reactions	Share
both	1,391	33.7%
eQuilibrator only	**364**	**8.8%**
shipped only	1,819	44.0%
neither	557	13.5%
union	3,574	86.5%

So the recomputation **gains 364 reactions the shipped file leaves empty**, and the union covers 86.5 percent, nearly nine points above what the GEM ships alone. The 1,819 it loses are the real cost, and the honest use of the two together is a merge that prefers the recomputed value where it exists, for its uncertainty, and falls back to the shipped point value elsewhere. That merge is not built.

3.3 The uncertainty, which is the point ✗

Figure 2 shows the standard error on the reactions whose energy is estimable. Two facts sit inside it. Of 1,673 estimable reactions, 888 have a standard error of exactly zero; those are transport reactions, where the same compound appears on both sides of the membrane and its uncertainty cancels term by term. The remaining 785 carry a genuine standard error with a median of 2.30 kJ mol^{-1} and a 95th percentile of 6.01.

Estimability is a separate question from uncertainty and is easy to conflate. Component contribution returns two sets of directions, one with finite variance and one with infinite variance, the latter being directions the training corpus never determined. Almost every individual compound has a nonzero infinite component, because an absolute formation energy is defined only up to element conservation. Those directions cancel in a mass-balanced reaction, which is exactly why the test must be applied to a reaction and never to a compound in isolation. A reaction whose infinite component does not cancel has an energy that is undefined rather than uncertain, and reporting it with a finite error bar would be the most misleading thing this table could do, so it is masked instead.

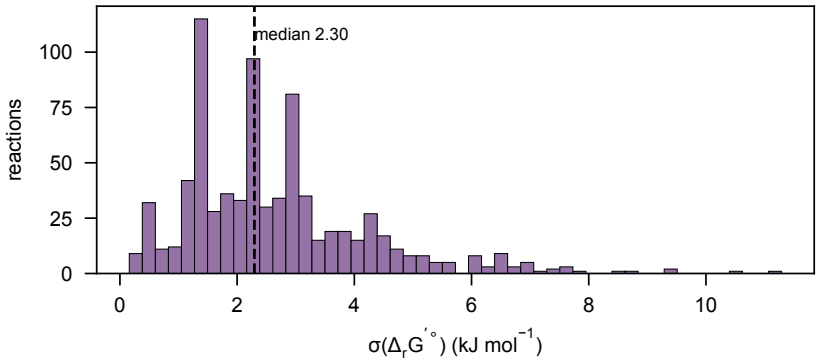


Figure 2: Standard error on $\Delta_r G'^{\circ}$ for the 785 estimable reactions whose uncertainty is nonzero, with the median marked. The 888 estimable reactions omitted here have a standard error of exactly zero because they are transport steps whose per-compound uncertainties cancel. The shipped table carries no uncertainty of any kind, so this quantity has no counterpart in it. Generated by `experiments/026-metabolism-flux/scripts/plot_equilibrator_thermo.py`.

3.4 Agreement with the shipped table ✗

Figure 3 compares the two tables on the 1,391 reactions both cover, at a uniform pH 7.0 so that the comparison is between methods rather than between pH assumptions.

The two classes of reaction must be read separately, and pooling them produces a misleadingly good number. At a single uniform pH a transport reaction moves a species between compartments with the same formation energy on both sides, so its energy here is exactly zero by construction while the shipped table carries a nonzero value for it. Those 840 reactions form the horizontal band at zero. On the 551 chemical reactions, which is where the two methods actually make comparable claims, the median absolute difference is **10.11 kJ mol^{-1}**. Pooling all 1,391 gives 5.60, and that smaller number is an artifact of the transport band rather than better agreement.

For scale, the shipped table disagrees with *itself* by a comparable amount: the reaction file and the reaction energies recomputed from its own metabolite file differ by a median of 9.53 kJ mol^{-1}. So the two methods agree about as well as the shipped table is internally consistent, which is the honest reading.

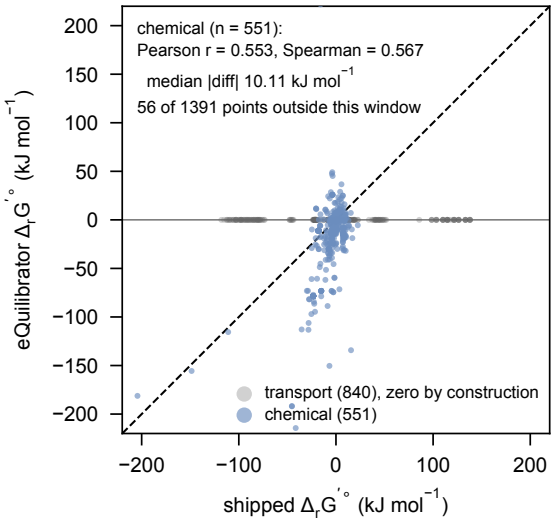


Figure 3: Reaction energies from the recomputation against the values yeast-GEM ships, on the 1,391 reactions both cover, at uniform pH 7.0, ionic strength 0.25 M and 303.15 K. Chemical reactions and transport reactions are separated because a transport step is exactly zero here by construction: at one pH the same species has the same formation energy in both compartments. The quoted median absolute difference is for the chemical subset. The dashed line is equality. Generated by `experiments/026-metabolism-flux/scripts/plot_equilibrator_thermo.py`.

3.5 Compartment pH is implemented and not trusted ✗

Applying a per-compartment pH rather than a uniform one moves 1,309 of 1,755 covered reactions, by a median of 42.2 kJ mol^{-1} and by as much as 1,062 (Figure 4a). Shifts of that size are not physically plausible for single reactions, so the uniform build is the artifact to use and the compartment build is experimental. The pH table itself is textbook organellar values, recorded in the output as `assumed`, not sourced from a mirrored paper.

The motivation for wanting it remains: a proton crossing a membrane between two pH values is the entire driving force of a transport step, and computing every compartment at one pH sets that force to zero by construction. Diagnosing the oversized shifts is open work.

3.6 The correctness check ✗

The table stores per-metabolite formation energies because that is what the flux layer consumes, and reaction energies follow as $\sum_i S_{ij} \Delta_f G'^{\circ}_i$. That identity is an assumption until tested: an error in the Legendre transform, in the compartment pH assignment, or in a stoichiometric sign would each produce a table that looks entirely reasonable and is wrong.

Pricing the same reactions both ways, by summing the table and by handing the resolved compounds to eQuilibrator's own `standard_dg_prime`, agrees to a maximum absolute difference of 9.1×10^{-13} kJ mol^{-1} over 40 single-compartment mass-balanced reactions (Figure 4b). Only single-compartment reactions are used, because a reaction spanning two compartments has no single pH and the API call would not be answering the same question; only balanced ones, because an unbalanced reaction carries an arbitrary offset.

As an absolute check that eQuilibrator is configured correctly before anything is compared against it, four textbook reactions price at pH 7.2, ionic strength 0.25 M and 303.15 K as: ATP hydrolysis -28.93 ± 0.30, phosphoglucose isomerase $+2.66 \pm 0.38$, fumarase -3.43 ± 0.28 and triosephosphate isomerase $+5.61 \pm 0.55$ kJ mol^{-1}.

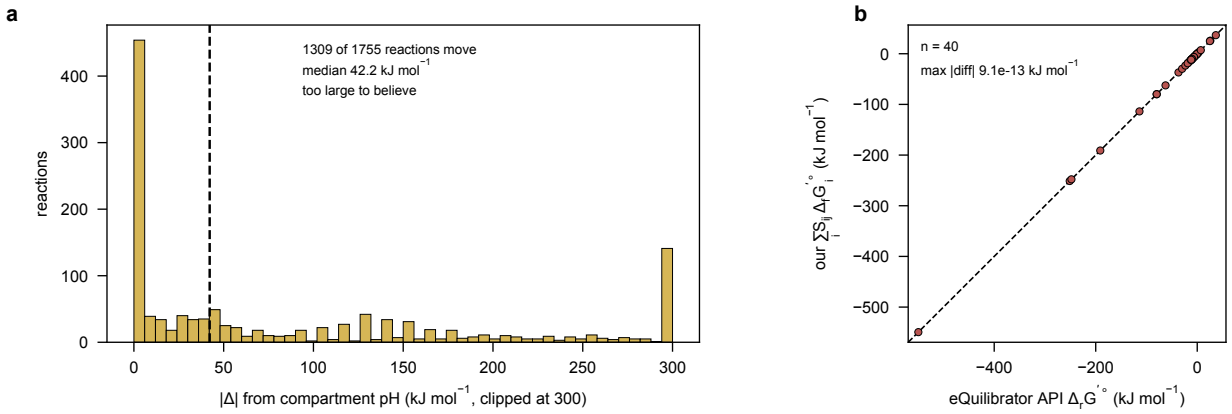


Figure 4: The two checks that decide whether the recomputed table is usable, which are the same question asked twice. **a**, Absolute change in $\Delta_r G'^{\circ}$ when each metabolite is transformed at its compartment's pH rather than at a uniform 7.0, over the reactions covered in both builds, clipped at 300 kJ mol^{-1} for display. The median shift of 42.2 kJ mol^{-1} is too large to be credible for ordinary reactions, which is why the uniform build is the one in use. **b**, Reaction energies computed by summing our per-metabolite table against the same reactions priced directly by eQuilibrator's `standard_dg_prime`, over 40 randomly chosen single-compartment mass-balanced reactions. Maximum absolute difference 9.1×10^{-13} kJ mol^{-1}, which is floating-point noise. Panel b is a correctness check rather than a result: had the points left the diagonal, nothing else in this section would mean anything. Generated by `experiments/026-metabolism-flux/scripts/validate_equilibrator_thermo.py` and `plot_equilibrator_thermo.py`.

3.7 Where the method stops ✗

The betaxanthin cassette is the test of the claim that this transfers to compounds no database has tabulated. Of five intermediates, all five match their declared molecular formula against a PubChem structure lookup, and two resolve to a formation energy: L-DOPA at -157.68 with $\sigma = 4.30$, and dopaquinone at -168.31 with $\sigma = 7.54$ kJ mol^{-1}, both from structure alone with no identifier required.

The other three, cyclo-DOPA, betanidin and betalamic acid, are absent from the compound cache, and the blocker is a *commercial license rather than a method*. Creating a new compound runs a group decomposition that needs pKa estimates, and eQuilibrator obtains those from ChemAxon's `cxcalc`; without that license `equilibrator_assets` loads in read-only mode and refuses to create compounds. So the generalization claim holds for structures the cache already knows and fails for genuinely new ones. Closing the gap needs a pKa route that does not pass through ChemAxon, not more plumbing.

4 Reproducing Wu et al.'s Fig. 3 ✗

4.1 What the original figure claims ✗

Wu et al. (2026) build Yeast-MetaTwin, a comprehensive metabolic network for *S. cerevisiae* obtained by expanding reaction rules over the yeast metabolome, and ask how the resulting *underground* reactions differ kinetically from the *core* network already in Yeast9. Their Fig. 3a–h answers it with eight empirical cumulative distributions, one per predictor and parameter: k_{cat} from DLKcat, UniKP, EITLEM-Kinetics, TurNuP and DeepEnzyme, then K_M from Boost_KM, UniKP and EITLEM-Kinetics. Their conclusion is that underground metabolism is separated from known metabolism by K_M rather than by k_{cat}.

The construction is taken from their own notebook: sort the values, plot the running proportion, and mark each curve's median with a short vertical tick. Their split is on whether a reaction identifier contains `rxn`, which is how their tables encode a predicted rather than a curated reaction.

4.2 The per-pair predictions behind Fig. 3 were not released ✗

The intention was to draw two things on each panel: the authors' own released predictions split core against underground, and our independent re-run over yeast-GEM. The first is not available.

Their notebook reads its tables from `Results/kcat_km_predict/`, naming each one explicitly:

- `yeast8U_kcat_predict.csv` DLKcat
- `yeast8U_unikp.csv` UniKP, both parameters
- `yeast8U_kcat_result_TurNuP.csv` TurNuP
- `yeast8U_km_km_predict.csv` Boost_KM

That directory is in neither released artifact. The GitHub repository ships `Results/` with nine subdirectories, none of them `kcat_km_predict`, and its `source_data` holds only `fig2-a`, `fig3-g`, `fig3-j`, `fig4-b`, `fig5-b`, `figs11` and `figS8`; the file named `fig3-g.csv` contains EC-number counts rather than a K_M distribution. The 6.6 GB Zenodo archive (10.5281/zenodo.13911783) contains exactly three top-level directories, `Data`, `Data_retrosynthesis` and `esm`, and of its 1,021,022 entries **zero** match `kcat`, `unikp`, `turnup`, `eitlem` or `deepenzyme`.

This is checkable and is stated as a fact about the release rather than as a criticism of the work: the Data Availability statement says data for reproducing all figures is available on GitHub and Zenodo, and for Fig. 3a–h it is not.

Two consequences follow. The **underground arm cannot be drawn**, because generating the Yeast-MetaTwin reaction set is a separate retrobiosynthesis pipeline that this work does not rebuild. And what remains is the stronger half of a reproduction anyway: the curves in Figure 5 are re-derived by running the same published models over the same organism from hash-pinned mirrors, rather than re-plotted from someone else's numbers.

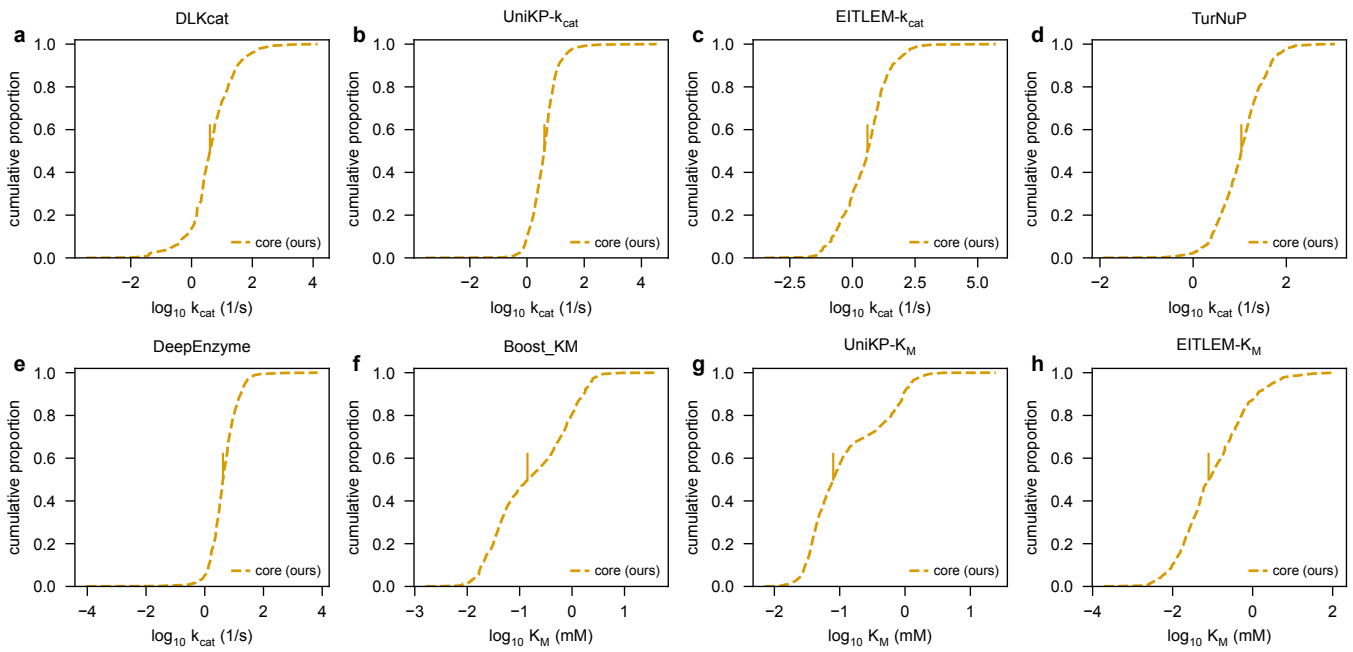


Figure 5: Empirical cumulative distributions of predicted kinetic parameters, following Wu et al. (2026) Fig. 3a–h. Panels a–e are k_{cat}, panels f–h are K_M; the predictor is named on each panel. Amber dashed is our independent run of the same predictor over yeast-GEM 9.0.2, which is the core network. The authors' core and underground curves are absent because the per-pair tables their notebook reads are in neither released artifact, as the text sets out. A short vertical tick marks each curve's median, as in the original. All eight predictors ran, so every panel carries a curve. Generated by `experiments/026-metabolism-flux/scripts/plot_kinetic_distributions.py`.

4.3 The log base is not a convention ✗

Each predictor was validated against something its own authors published, and the exercise paid for itself. **Two of these models exponentiate base 2 and two base 10**, and reading either wrong produces turnover numbers that look entirely credible.

Table 3: What each predictor was checked against before its output was believed, and the exponentiation base that check established.

Predictor	Checked against	Base
DLKcat	3 released examples, to 4 decimals	$\log_2$
DeepEnzyme	137/137 checkpoint tensors match; 3 independent base checks	$\log_2$
EITLEM	README worked example, 1.3904 vs a stated 1.39	$\log_{10}$
TurNuP	tutorial value, 216.114853 vs 216.114853	$\log_{10}$

Two blockers in an earlier audit also dissolved on inspection. DeepEnzyme's NVIDIA apex requirement is listed in `requirements.txt` and imported nowhere in its code, so it was a training-time mixed-precision dependency and the network runs unmodified. TurNuP's weights exist but are split across two Zenodo records, because the enzyme half needs an ESM-1b fine-tune that ships only with the prediction-function archive.

4.4 The comparison between predictors, which the original does not make ✗

Reading the panels side by side invites a question the paper does not ask: do the predictors agree with each other? Four of the five agree on the median k_{cat} to within 4.0 to 4.2 s^{-1}, and that agreement is superficial. Their *per-pair* rank correlation, on identical enzyme-substrate pairs, ranges from **0.015 to 0.509**.

Table 4: Agreement between predictors on identical enzyme-substrate pairs, in $\log_{10}$ space. The last column is the median absolute difference in decades. Produced by `experiments/026-metabolism-flux/scripts/compare_kinetic_predictors.py`.

Pair	n	Pearson	Spearman	Med. |diff|
EITLEM vs UniKP	7,351	0.560	0.509	0.51
TurNuP vs UniKP	5,038	0.458	0.433	0.44
EITLEM vs TurNuP	5,143	0.361	0.330	0.64
DLKcat vs UniKP	7,351	0.337	0.224	0.46
DeepEnzyme vs DLKcat	7,300	0.270	0.284	0.40
DLKcat vs EITLEM	7,351	0.248	0.169	0.70
DLKcat vs TurNuP	5,038	0.142	0.084	0.56
DeepEnzyme vs TurNuP	5,035	0.111	0.094	0.51
DeepEnzyme vs UniKP	7,300	0.109	0.015	0.40
DeepEnzyme vs EITLEM	7,300	0.090	0.025	0.68
K_M: Boost_KM vs UniKP	7,157	0.764	0.789	0.29
K_M: EITLEM vs UniKP	7,351	0.463	0.406	0.50
K_M: Boost_KM vs EITLEM	7,157	0.359	0.345	0.54

That matters for how Fig. 3 should be read. A shift between two curves within one panel is interpreted as a property of underground metabolism; the spread *between* panels, on the same pairs, is a factor of three to five. An effect smaller than that sits inside the disagreement between the instruments measuring it, which is consistent with the paper's own conclusion being framed on K_M, where the separation is large, rather than on k_{cat}, where it is small.

TurNuP is the exception to the median agreement, at 10.8 s^{-1}, and its coverage is 55.2 percent rather than 95. Both follow from the same property: it consumes the *whole reaction*, and the authors' code invalidates a reaction as soon as one participant lacks a structure. That excludes 1,147 of 2,540 reactions, almost all acyl-chain-resolved lipids.

4.5 The comparison per gene, which is the unit the model reads ✗

Table 4 is keyed by enzyme-substrate pair. That is the right key for asking whether two models predict the same number for the same measurement, and it is not the key the flux layer reads: a catalytic unit draws one k_{cat} per gene. So the question that decides whether a predicted table is usable is how far the choice of predictor moves *that gene's* value. Figure 6 answers it, taking each gene's value as the median over its pairs in $\log_{10}$ space, separately per predictor.

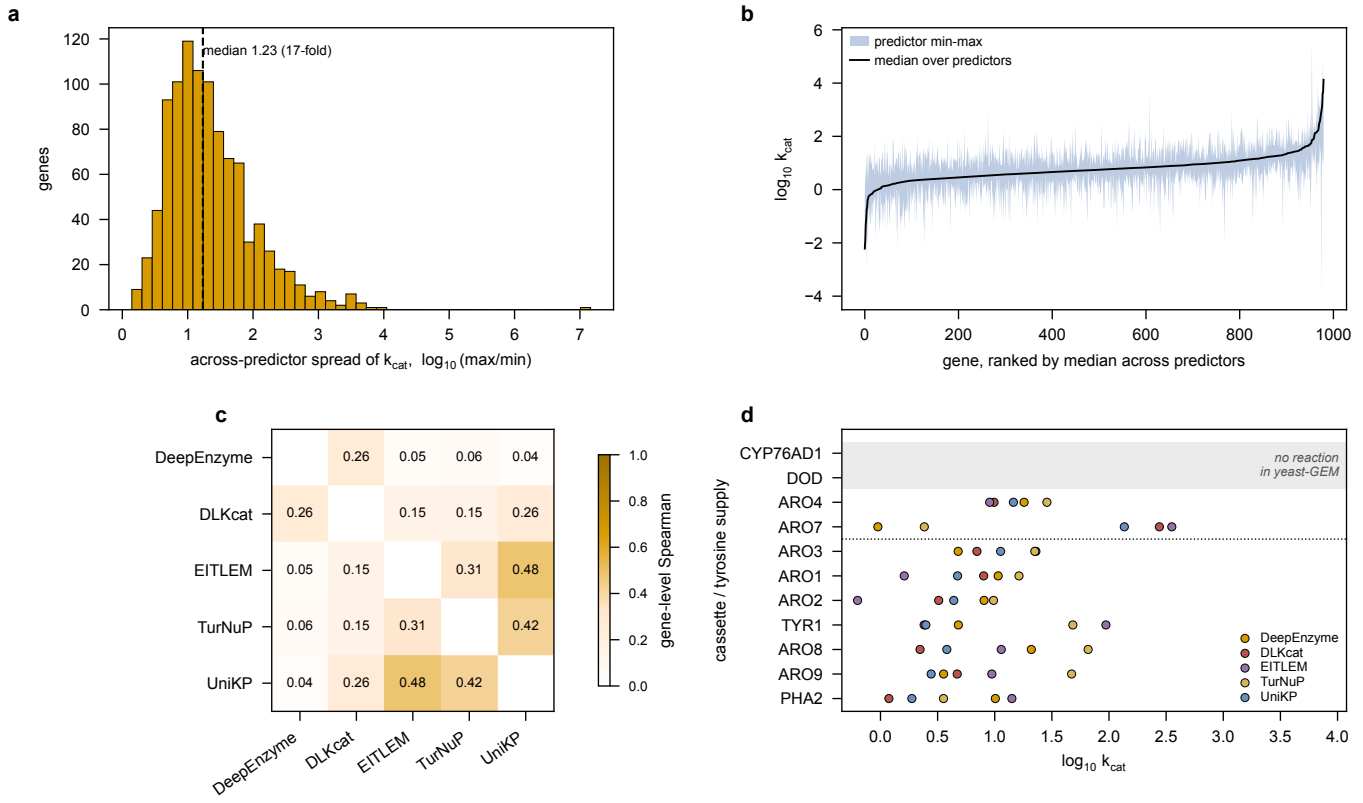


Figure 6: Agreement between the five k_{cat} predictors at the level of a gene, over the 980 genes all five cover. **a**, Across-predictor spread per gene, the difference between the highest and lowest predictor in decades; the dashed line is the median, 1.23. **b**, The same spread against the dynamic range of k_{cat} itself, genes ranked by their median across predictors: the band is the predictor min-max, the line the median. **c**, Gene-level Spearman between predictors, on the white-to-orange scale 020 uses for its comparison matrices; the diagonal is left blank because a predictor against itself is 1 by construction. **d**, The genes the betaxanthin case study runs on. Above the dotted rule is the Btx cassette, below it the aromatic-amino-acid genes that supply its tyrosine precursor. `CYP76AD1` and `DOD` are heterologous and have no reaction in yeast-GEM, so no predictor emits a value for them; the shaded band carries that once rather than repeating it per gene. Generated by `experiments/026-metabolism-flux/scripts/plot_per_gene_predictor_agreement.py`.

For k_{cat} the disagreement between predictors is larger than the quantity being predicted. The median gene moves 1.23 decades between the highest and lowest predictor, a factor of 17, and 67 percent of genes move by more than a full decade. The 10th-to-90th-percentile range of the consensus estimate across genes is 0.94 decades. The ratio is 1.32: choosing a different published predictor moves one gene's turnover number further than real enzymes differ from one another. Panel b is that statement drawn, and the band never narrows.

For K_M it is not. The same construction over the three K_M predictors gives a median spread of 0.70 decades against a consensus range of 1.34, a ratio of 0.52, and Boost_KM agrees with UniKP at a gene-level Spearman of 0.75, higher than any pair of k_{cat} predictors reaches. Both facts are agreement rather than accuracy, since almost none of these pairs has a measured value. Read that way they arrive at Wu et al.'s conclusion from the other side: their claim is that underground metabolism separates from known metabolism on K_M rather than on k_{cat}, and K_M is also the parameter the instruments agree on well enough for a separation to be legible.

The consequence for the flux layer. An enzyme-constrained bound $v_j \leq k_{cat,j} E_j$ inherits panel a as its error bar, so a single predictor's table should not be treated as the capacity constraint. The registry in `torchcell/datasets/kinetics/` exists for that reason: predictor choice is a hyperparameter with a measured effect size, not a settled detail.

5 Where these tables enter the model ✗

Figure 7 places the flux module inside the cell graph transformer. The module is neither of the two instrument classes the architecture previously had: a Type I instrument maps a representation to a representation and a Type II instrument maps a representation to a measured output, while the flux module maps a representation to a *flux vector*, a latent physical state from which phenotypes are then read. It sits between them.

Three of its inputs are parameter tables rather than learned weights, and those three are what this document builds. The k_{cat} table sets the upper bound of the box that the flux is parameterized inside, so it fixes what fluxes are reachable at all. The formation energies set the reaction potential whose sign the second-law penalty acts on, which is what makes that penalty a physical statement rather than a regularizer. The K_M table is built and not yet enabled, for the reason Section 6 gives.

5.1 Why these two tables, in two equations ✗

Each table earns its place by removing a specific failure the stoichiometry alone cannot.

k_{cat} is what lets a genotype change a flux. In one line, with the gene-protein-reaction fold abstracted into $c_j \in [0, 1]$:

$$|\nu_j| \;\le\; \underbrace{c_j\big(H_{\text{pert}}\big)}_{\text{GPR availability}} \cdot \underbrace{k_{cat,j}\,\bar{E}}_{\text{capacity}}. \tag{1}$$

c_j is not a pooled function of every gene. Each GEM gene is gated on its own embedding, $\gamma_g = \sigma(w^\top h_g)$, with a deletion pinning $\gamma_g = 0$; those gates are folded to the reaction by a softmin over the subunits of a complex and a sum over isozymes. A reaction with no GPR keeps $c_j = 1$, since an unannotated reaction is one nobody assigned a gene to rather than one no gene catalyzes.

$\bar{E}$ is a constant, and saying so matters. It is a nominal enzyme abundance, 10^{-3}, not a learned output and not a measured proteome, so the ceiling reduces to $3.6\,k_{cat,j}\,c_j$ per catalytic unit once k_{cat} is converted to h^{-1}. $\bar{E}$ sets the *scale* of the bound; everything genotype-dependent enters through c_j. Reading $\bar{E}$ as a per-gene enzyme level would overstate what the layer does: **no enzyme abundance is predicted anywhere in it.**

The protein budget does not change that, because its e_g runs the other way. It is the enzyme demand *implied* by a flux, $e_g = \sum_{u \ni g} |\nu_{\rho(u)}|/k_{cat,u}$, which is then priced against a budget as $\big[\sum_g MW_g e_g / P_{\text{avail}} - 1\big]_+$. So k_{cat} appears twice and in opposite directions: it caps a flux in Equation (1), and it converts a flux back into a protein cost here. Neither is a proteome measurement.

This is the obvious place for one. The layer already has the hook, and a measured proteome exists in the same project: Zelezniak's 726 proteins across 97 kinase deletions. Replacing the constant with a predicted or measured E_g is what would turn the capacity bound from a scale into a quantity.

Without Equation (1) the only limits on ν_j are yeast-GEM's own bounds, and those carry no capacity at all: **4,129 of 4,131 reactions sit at** ± 1000 **or** 0, which encodes direction and on-off and nothing else. A gene deletion would then have no route to change the *magnitude* of any flux. k_{cat} is the unit conversion from an amount of enzyme to a ceiling on rate, and this product is the only path by which a genotype reaches the network's capacity.

$\Delta_r G$ enters as a term in the loss, not as a constraint. There is no solver and no binary variable, so the second law is imposed by making its violation expensive. The trained objective is

$$\mathcal{L} = \mathcal{L}_{\text{data}} \;+\; \lambda_{\text{th}} \underbrace{\frac{1}{|\mathcal{J}|} \sum_{j \in \mathcal{J}} \frac{\big[\nu_j\, \Delta_r G_j + \epsilon\big]_+}{|\nu_j|\,|\Delta_r G_j| + \epsilon}}_{C_{\text{th}}}, \qquad \Delta_r G_j = \Delta_r G'^{\circ}_j + RT \sum_{i \neq \mathrm{H}^+} S_{ij} \ln c_i, \tag{2}$$

where $[\,\cdot\,]_+$ is the hinge and $\mathcal{J}$ runs over reactions with a known $\Delta_r G$, excluding exchange, biomass and water transport. The tabulated formation energies supply $\Delta_r G'^{\circ}_j$, which is the one place this document's thermodynamic table reaches the model.

The normalization is load-bearing, not cosmetic. The raw hinge $[\nu_j \Delta_r G_j]_+$ carries units and, measured at initialization on a real batch, is about **72 against a data loss of about 1**. An unnormalized term of that size does not regularize the model, it replaces it. Dividing by $|\nu_j||\Delta_r G_j|$ caps each reaction's contribution at 1 and turns C_{th} into the fraction of the available driving force running uphill, which is also the quantity worth reporting.

Why it earns its place: because $\Delta_r G$ is a difference of potentials it sums to zero around any cycle, so no cycle can run downhill at every step, and Equation (2) therefore penalizes internal loops. Capacity does not do this. Requiring growth at 90 percent of optimum narrows 2,316 reactions, 56.1 percent of the network, and leaves **427 reactions still free to take any value in** ± 1000, all of them internal loops. Only the thermodynamic term removes those, and it does so with no binary variable, which is what a loopless flux balance program needs one for.

The heterologous cassette enters as a typed perturbation on the genome-scale model, which is what puts a betaxanthin pathway and a gene deletion inside the same object rather than in two parallel code paths.

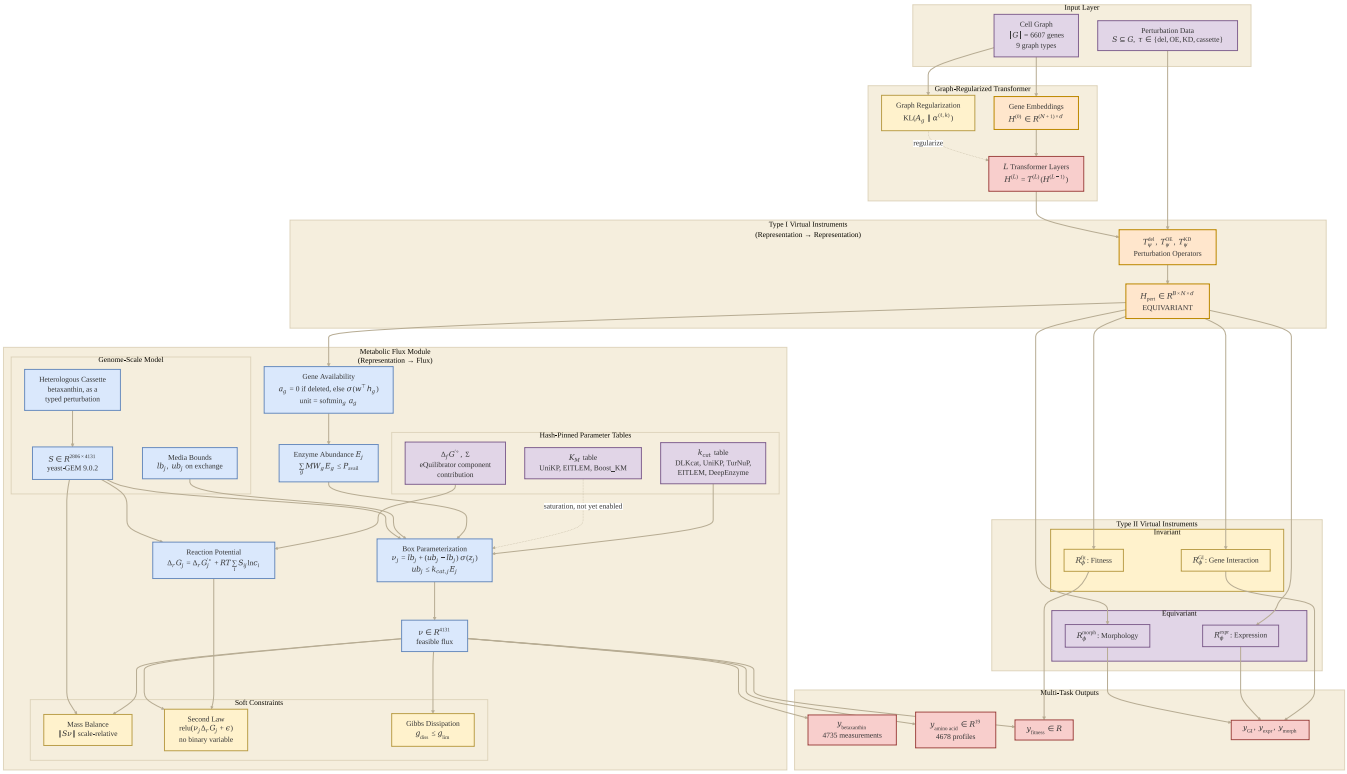


Figure 7: The cell graph transformer with the enzyme-constrained thermodynamic flux module. Purple marks inputs and the hash-pinned parameter tables built here; orange the embeddings and the Type I perturbation operators; red the transformer layers and the outputs; yellow the graph regularization, the invariant readouts and the soft constraints; blue the metabolic module, whose color slot was unused in the previous version of this diagram so the new content separates at a glance. Every hard constraint is either exact by construction, as the box and the capacity bound are, or relaxed to a smooth penalty; nothing is enforced by a binary variable, which is what makes the layer differentiable and what removes the mixed-integer cost a per-genotype thermodynamic program pays. The dashed K_M edge marks a table that exists but is not yet consumed. Scaled to the text block: the source renders at 211.7 mm wide against a 182 mm block, so this is not shown at true print size. Source: `notes/torchcell.models.equivariant_cell_graph_transformer.mermaid.metabolic-module.md`, rendered by `notes/assets/publish/scripts/mermaid_pdf.sh`.

6 What these tables cannot support ✗

6.1 A predicted turnover number is not a measured one ✗

Every row in the k_{cat} and K_M builds is a model output. The tables record that in a provenance column beside the value, because a number with no provenance becomes indistinguishable from a measurement within two steps, and the coverage gate has to be able to separate an experimental value, a predicted value and a filled default.

The size of the distinction is measurable. Two predictors trained on overlapping data, applied to the same 7,351 pairs, disagree by a median factor of about three. Neither model's own reported error is that large. The disagreement is the honest uncertainty on any single value, and it is wider than any published accuracy figure would suggest, because those figures are computed on held-out data drawn from the same corpus rather than on a genome-scale application.

6.2 Accuracy against the Open Enzyme Database is memorization, not generalization ✗

Comparing predictions to the Open Enzyme Database looks like validation and is not. The database aggregates BRENDA and Sabio-RK, which is what these models were trained on, so a matched enzyme-substrate pair is almost certainly inside their training sets. Numbers from that comparison are reported here explicitly as memorization checks.

The join also has to be on the *pair*. Joining on enzyme alone compares a prediction for one substrate against a measurement for another, and doing so moves one of these correlations from 0.98 to 0.82 and another from 0.62 to 0.12. The looser join is the one that is easy to write by accident.

6.3 K_M is built and deliberately not consumed ✗

The flux layer does not use K_M. The parameter enters only through a saturation factor $\eta_{\text{sat}} = \prod_i c_i/(K_{M,i} + c_i)$, which needs intracellular metabolite concentrations, and those are measured for 191 of 2,806 metabolites, 6.8 percent. Adding K_M today would add a parameter and a free variable per reaction with no anchor for either.

The measured argument for adding it anyway is promiscuity: underground reactions show roughly twice the K_M at indistinguishable k_{cat}, so a k_{cat}-only model cannot tell a promiscuous side activity from a native one. The measured argument against is that the K_M predictions here are the least reliable of the set, and treating them as constants rather than as order-of-magnitude priors would put that unreliability directly into a constraint.

6.4 The thermodynamic gaps ✗

Three, in order of how much they bind.

Novel compounds need a licensed tool. Adding a compound eQuilibrator has never seen requires pKa estimates from ChemAxon's `cxcalc`. Without the license the compound cache is read-only, so the betaxanthin intermediates that are not already in it cannot be priced at all. This is the binding constraint on applying the method to heterologous pathways, which is exactly where it is most wanted.

Coverage fell. Reaction coverage went from 77.7 percent to 42.5 percent, because a reaction needs every participant resolved. The unresolved metabolites are dominated by acyl-CoA isomers and combinatorial lipid species, which are chemically ordinary and merely un-indexed under the names the model uses.

Compartment pH is not validated. It is implemented, it produces shifts too large to believe, and the pH table is assumed rather than sourced. Until that is diagnosed the uniform build is the one to use, which means transport thermodynamics is set to zero by construction and the proton motive force is absent from the model.

6.5 What the mirror records when a predictor cannot run ✗

All six now run. Where a predictor could not be run at all, the mirror records `runnable: false` with the specific blocker, so that a gap in coverage is never mistaken for a modeling choice.

Boost_KM carried that flag until its weights were read rather than its filenames, and the correction is recorded in Section 1 rather than quietly dropped: the repository does ship fitted regressors, and the obstacle was never a missing model. It was that UniRep is a TensorFlow 1 mLSTM stepping one residue at a time on CPU, which on a first attempt occupied roughly 85 cores. The ESM-1b variant Wu et al. actually used replaces that stage with a transformer that embeds all 887 enzymes on one GPU in four minutes.

7 Every source, for citation and for the mirror ✗

Each predictor is a published method, a code repository at a pinned commit, and a weights archive, and the three are cited differently. The tables below carry all of them so a reference can be added to the library and a manifest field filled without re-deriving the provenance. Every commit and `sha256` comes from `$DATA_ROOT/models/kinetics/<name>/manifest.json` or from `mirror_kinetic_models.py`; the DOIs were verified against Crossref rather than recalled.

7.1 Papers ✗

Table 5: The publication behind each artifact used in this document.

Tool	Paper	DOI
DLKcat	Li et al., Deep learning-based k_{cat} prediction enables improved enzyme-constrained model reconstruction. *Nat Catal* 5:662–672, 2022	`10.1038/s41929-022-00798-z`
UniKP	Yu et al., UniKP: a unified framework for the prediction of enzyme kinetic parameters. *Nat Commun* 14:8211, 2023	`10.1038/s41467-023-44113-1`
EITLEM	Shen et al., EITLEM-Kinetics: a deep-learning framework for kinetic parameter prediction of mutant enzymes. *Chem Catal* 4:101094, 2024	`10.1016/j.checat.2024.101094`
TurNuP	Kroll et al., Turnover number predictions for kinetically uncharacterized enzymes using machine and deep learning. *Nat Commun* 14:4139, 2023	`10.1038/s41467-023-39840-4`
DeepEnzyme	Wang et al., DeepEnzyme: a robust deep learning model for improved enzyme turnover number prediction. *Brief Bioinform* 25(5):bbae409, 2024	`10.1093/bib/bbae409`
Boost_KM	Kroll et al., Deep learning allows genome-scale prediction of Michaelis constants from structural features. *PLoS Biol* 19:e3001402, 2021	`10.1371/journal.pbio.3001402`
eQuilibrator	Beber et al., eQuilibrator 3.0: a database solution for thermodynamic constant estimation. *Nucleic Acids Res* 50:D603–D609, 2022	`10.1093/nar/gkab1106`
Component contribution	Noor et al., Consistent estimation of Gibbs energy using component contributions. *PLoS Comput Biol* 9:e1003098, 2013	`10.1371/journal.pcbi.1003098`
Yeast-MetaTwin	Wu et al., Systematically exploring yeast metabolism through retrobiosynthesis and deep learning. *Nat Catal* 9:434–447, 2026	`10.1038/s41929-026-01523-w`

7.2 Code and weights ✗

Table 6: What was mirrored, at which revision. A commit is given in full because a short hash is not a pin.

Tool	Repository, at commit	Weights
DLKcat	`SysBioChalmers/DLKcat` `7c15d0d4a7ac029f9d75564d9f2a93874aeaaec7`	in repo, extensionless `state_dict`
UniKP	`Luo-SynBioLab/UniKP` `3ad5576aaa2c8c0dd0e0b6c283c1d365ab23c6ea`	HuggingFace `HanselYu/UniKP`
EITLEM	`XvesS/EITLEM-Kinetics` `7bb9d6668d86263f17dddc1915fccff5712fd511`	Zenodo `10.5281/zenodo.16153803`, 12.3 GB
TurNuP	`AlexanderKroll/kcat_prediction` `98d88a37b66f64381086b5492f315e763c528fa0`	Zenodo `10.5281/zenodo.8367052` and `10.5281/zenodo.8038678`
DeepEnzyme	`hongzhonglu/DeepEnzyme` `d225ec31a1079177eea38fcf44c5c4b7afe516d8`	figshare `10.6084/m9.figshare.25771062.v2`
Boost_KM	`AlexanderKroll/KM_prediction` `432a4fc02afa20fcb8296fb96ce035a99ae33e85`	in repo; the ESM-1b variant is vendored in the Yeast-MetaTwin release
ESM-1b	Meta `fair-esm`, base checkpoint	`esm1b_t33_650M_UR50S.pt`, sha256 `0569754e...83a8b1`
Yeast-MetaTwin	`LiLabTsinghua/Yeast-MetaTwin` `de1268eea271a67ffb0581aab51b3549c8020950`	Zenodo `10.5281/zenodo.13911783`, 6.6 GB

7.3 Four provenance defects to repair ✗

These are recorded rather than fixed silently, because each one breaks the rebuild guarantee in a different way.

- **EITLEM's recorded source URL is dead.** The manifest holds `XvQiao/EITLEM-Kinetics`, which now returns 404; the repository was renamed to `XvesS/EITLEM-Kinetics`. The pinned commit still resolves there, so the mirrored bytes are fine and only the retrieval command needs updating.

- **DeepEnzyme's retrieval command is a placeholder.** It contains the literal text `<figshare 10.6084/m9.figshare.25771062.v2>` rather than a URL, so that one file cannot be re-fetched by running the record.
- **eQuilibrator and Yeast-MetaTwin have no manifest.** Both sit outside `models/kinetics/`. The MetaTwin commit had to be read out of `.git`, and `metatwin.rar` is pinned by size alone with no `sha256`.
- **All six kinetics manifests carry `citation_key: null`.** Once these papers are in the mirror, that field joins each weight file to its publication, which is what makes a predicted value citable rather than merely traceable.

One extraction hazard is worth recording alongside them. `bsdtar` reading `esm1b_t33_650M_UR50S.pt` out of the 6.6 GB RAR **exited 0 and wrote 649,783,100 bytes** where the archive's own listing says 7,828,576,466. The truncation is silent, and a checkpoint that short still opens far enough to look like a file. It was caught by comparing against the listed size and then confirmed by `torch.load` refusing it. The bytes finally used were fetched from the published `fair-esm` URL and verified to the recorded `sha256`, which is the general lesson: a size or hash check is what makes an extractor's exit code mean anything.